

22. live forecast 取得ノード

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 rclpy、rcl、rmw、DDS/RTPS の層	7
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	8
6.11 rqt_graph、ros2 CLI、rosviz をいつ使うか	8
6.12 天候、route、補間、時刻、単位	9
6.13 freshness、filter、guard、fallback、fail-safe	9
6.14 制御、状態、入力、モデル予測制御 MPC	9

7 関数・クラスを上から順に解説	10
7.1 L15 クラス WeatherFetchNode	10
7.2 L16 関数 WeatherFetchNode.__init__	11
7.3 L104 関数 WeatherFetchNode._on_gps	13
7.4 L111 関数 WeatherFetchNode._fetch_once	14
7.5 L138 関数 main	16
7.6 設定値と入力パラメータ	17
7.7 ROS topic I/O	17
8 処理の流れ	17
9 このファイルの位置づけ	17

1 対象

項目	内容
ファイル	mpc_solarcar/weather_fetch_node.py
ソース SHA-256	bef650eab4797b7c832c841ba7eaa271ab3cccec171f9b270c69172f9835bb50
種別	Python
区分	runtime node
役割	chase GPS または fallback 座標から Open-Meteo forecast を取得し、planner が読む CSV を更新する。
起動文脈	live 系の forecast 更新入口。

2 このファイルを読む理由

chase GPS または fallback 座標から Open-Meteo forecast を取得し、planner が読む CSV を更新する。

- raw forecast CSV を定期更新する。
- planner 自体は topic ではなく forecast CSV を読む。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/live_launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/weather_utils.py
- mpc_solarcar/wind_correction_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py
- mpc_solarcar/weather_utils.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from .weather_utils import fetch_openmeteo_forecast, write_forecast_csv

class WeatherFetchNode(Node):
    def __init__(self) -> None:
        super().__init__("weather_fetch_node")
```

```

self.declare_parameter("profile_yaml", "")
self.declare_parameter("output_csv", "")
self.declare_parameter("raw_forecast_csv", "")
self.declare_parameter("gps_topic", "/chase/gps")
self.declare_parameter("fetch_period_sec", 3600.0)
self.declare_parameter("forecast_days", 3)
self.declare_parameter("step_minutes", 10)
self.declare_parameter("timezone_name", "Australia/Darwin")
self.declare_parameter("fallback_latitude", -12.4634)
self.declare_parameter("fallback_longitude", 130.8456)
self.declare_parameter("tcell_gain", 0.03)

profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
if profile_yaml:
    profile_path, cfg = load_profile(profile_yaml)
    runtime_cfg = get_section(cfg, "runtime")

```

5 主要構造の読み取り

主要クラスは WeatherFetchNode。主要関数は main。ROS パラメータ宣言は 11 件。ROS I/O は publisher 1 件、subscription 1 件。

5.1 主要クラス・関数

行	種別	名前	補足
15	class	WeatherFetchNode	bases: Node
16	function	__init__	args: self
104	function	_on_gps	args: self, msg
111	function	_fetch_once	args: self
138	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
15	クラス WeatherFetchNode を定義する。
138	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	from __future_ _import annotations	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	from pathlib import Path	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L85, L86。
5	import rclpy	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L39, L40, L44, L48, L52, L56, L60, L69, ...。

6	<code>fromrclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L15。
8	<code>fromsensor_msgs. msgimportNavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L99, L104。
9	<code>fromstd_msgs. msgimportString</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L98, L130, L134。
11	<code>from.solar_ profileimportget_path,get_ section,load_profile</code>	profile YAML 読込と検証から <code>get_path</code> , <code>get_section</code> , <code>load_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L32, L33, L34, L35, L39。
12	<code>from.weather_ utilsimportfetch_openmeteo_ forecast,write_forecast_csv</code>	<code>weather_utils.py</code> から <code>fetch_openmeteo_forecast</code> , <code>write_forecast_csv</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/weather_utils.py</code> 。このファイル内での主な使用位置は L115, L123, L128。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self._step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```


rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \  
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \  
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \  
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status  
  
ros2 bag info outputs/bags/preflight  
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- ROS 2 Humble 公式: [rqt_graph](#)
- ROS 2 Humble 公式: [Recording and playing back data](#)
- ROS 2 Humble 公式: [Understanding nodes](#)

6.12 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.13 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.14 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPCを使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L15 クラス WeatherFetchNode

- 行範囲: L15–L135
- 所属: module 直下
- 定義: `WeatherFetchNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_gps` を定義する。
3. 関数 `_fetch_once` を定義する。

代表コード断片

```
class WeatherFetchNode(Node):
    def __init__(self) -> None:
        super().__init__("weather_fetch_node")
        self.declare_parameter("profile_yaml", "")
        self.declare_parameter("output_csv", "")
        self.declare_parameter("raw_forecast_csv", "")
        self.declare_parameter("gps_topic", "/chase/gps")
        self.declare_parameter("fetch_period_sec", 3600.0)
        self.declare_parameter("forecast_days", 3)
        self.declare_parameter("step_minutes", 10)
        self.declare_parameter("timezone_name", "Australia/Darwin")
```

```

self.declare_parameter("fallback_latitude", -12.4634)
self.declare_parameter("fallback_longitude", 130.8456)
self.declare_parameter("tcell_gain", 0.03)

profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
if profile_yaml:
    profile_path, cfg = load_profile(profile_yaml)
    runtime_cfg = get_section(cfg, "runtime")
    live_cfg = get_section(cfg, "live")
    weather_cfg = get_section(live_cfg, "weather")
    if not self.get_parameter("output_csv").value:
        self.set_parameters(
            [
                rclpy.parameter.Parameter("output_csv", value=get_path(cfg,
profile_path, "forecast_csv")),
                rclpy.parameter.Parameter(
                    "raw_forecast_csv",
                    value=str(weather_cfg.get("raw_forecast_csv", "") or ""),
                ),
                rclpy.parameter.Parameter(
                    "gps_topic",
                    value=str(weather_cfg.get("gps_topic", "/chase/gps")),
                ),
                rclpy.parameter.Parameter(
                    "fetch_period_sec",
...

```

7.2 L16 関数 WeatherFetchNode.__init__

- 行範囲: L16–L102
- 所属: WeatherFetchNode
- 定義: `__init__(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Parameter`, `Path`, `__init__`, `_fetch_once`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `expanduser`, `float`, `get`, `get_logger`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `cfg`, `live_cfg`, `profile_path`, `profile_yaml`, `raw_out`, `runtime_cfg`, `weather_cfg`
- 読み取る主なインスタンス属性: `self._fetch_once`, `self.on_gps`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.fetch_period_sec`, `self.get_logger`, `self.get_parameter`, `self.gps_topic`, `self.output_csv`, `self.set_parameters`
- 更新する主なインスタンス属性: `self.fallback_lat`, `self.fallback_lon`, `self.fetch_period_sec`, `self.forecast_days`, `self.gps_topic`, `self.latest_lat`, `self.latest_lon`, `self.output_csv`, `self.pub_status`, `self.raw_forecast_csv`, `self.step_minutes`, `self.tcell_gain`, `self.timer`, `self.timezone_name`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `profile_yaml` に `str(self.get_parameter('profile_yaml').value or '').strip()` の結果を代入する。
14. 条件 `profile_yaml` を判定し、真なら内部処理を行う。
15. `(profile_path, cfg)` に `load_profile(profile_yaml)` の結果を代入する。
16. `runtime_cfg` に `get_section(cfg, 'runtime')` の結果を代入する。
17. `live_cfg` に `get_section(cfg, 'live')` の結果を代入する。
18. `weather_cfg` に `get_section(live_cfg, 'weather')` の結果を代入する。
19. 条件 `not self.get_parameter('output_csv').value` を判定し、真なら内部処理を行う。
20. `self.set_parameters(...)` を実行する。
21. `raw_out` に `str(self.get_parameter('raw_forecast_csv').value or '').strip()` の結果を代入する。
22. `self.output_csv` に `Path(str(self.get_parameter('output_csv').value or ''))`.`expanduser()` の結果を代入する。
23. `self.raw_forecast_csv` に `Path(raw_out).expanduser()` if `raw_out` else `None` の結果を代入する。
24. `self.gps_topic` に `str(self.get_parameter('gps_topic').value)` の結果を代入する。
25. `self.fetch_period_sec` に `max(60.0, float(self.get_parameter('fetch_period_sec').value))` の結果を代入する。
26. `self.forecast_days` に `max(1, int(self.get_parameter('forecast_days').value))` の結果を代入する。
27. `self.step_minutes` に `max(1, int(self.get_parameter('step_minutes').value))` の結果を代入する。
28. `self.timezone_name` に `str(self.get_parameter('timezone_name').value or 'Australia/Darwin')` の結果を代入する。
29. `self.fallback_lat` に `float(self.get_parameter('fallback_latitude').value)` の結果を代入する。
30. `self.fallback_lon` に `float(self.get_parameter('fallback_longitude').value)` の結果を代入する。
31. `self.tcell_gain` に `float(self.get_parameter('tcell_gain').value)` の結果を代入する。
32. `self.latest_lat` に `None` の結果を代入する。

33. self.latest_lon に None の結果を代入する。
34. self.pub_status に self.create_publisher(String, '/weather/fetch_status', 10) の結果を代入する。
35. self.create_subscription(...) を実行する。
36. self.timer に self.create_timer(self.fetch_period_sec, self._fetch_once) の結果を代入する。
37. self.get_logger().info(...) を実行する。
38. self._fetch_once(...) を実行する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("weather_fetch_node")
    self.declare_parameter("profile_yaml", "")
    self.declare_parameter("output_csv", "")
    self.declare_parameter("raw_forecast_csv", "")
    self.declare_parameter("gps_topic", "/chase/gps")
    self.declare_parameter("fetch_period_sec", 3600.0)
    self.declare_parameter("forecast_days", 3)
    self.declare_parameter("step_minutes", 10)
    self.declare_parameter("timezone_name", "Australia/Darwin")
    self.declare_parameter("fallback_latitude", -12.4634)
    self.declare_parameter("fallback_longitude", 130.8456)
    self.declare_parameter("tcell_gain", 0.03)

    profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
    if profile_yaml:
        profile_path, cfg = load_profile(profile_yaml)
        runtime_cfg = get_section(cfg, "runtime")
        live_cfg = get_section(cfg, "live")
        weather_cfg = get_section(live_cfg, "weather")
        if not self.get_parameter("output_csv").value:
            self.set_parameters(
                [
                    rclpy.parameter.Parameter("output_csv", value=get_path(cfg,
profile_path, "forecast_csv")),
                    rclpy.parameter.Parameter(
                        "raw_forecast_csv",
                        value=str(weather_cfg.get("raw_forecast_csv", "") or ""),
                    ),
                    rclpy.parameter.Parameter(
                        "gps_topic",
                        value=str(weather_cfg.get("gps_topic", "/chase/gps")),
                    ),
                    rclpy.parameter.Parameter(
                        "fetch_period_sec",
                        value=float(weather_cfg.get("fetch_period_sec", 3600.0)),
                    ),
                ]
            )
    ...
```

7.3 L104 関数 WeatherFetchNode._on_gps

- 行範囲: L104–L109
- 所属: WeatherFetchNode
- 定義: _on_gps(self, msg: NavSatFix) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float

- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self.latest_lat`, `self.latest_lon`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `self.latest_lat` に `float(msg.latitude)` の結果を代入する。
3. `self.latest_lon` に `float(msg.longitude)` の結果を代入する。
4. `Exception` を捕捉した場合:
5. を返す。

代表コード断片

```
def _on_gps(self, msg: NavSatFix) -> None:
    try:
        self.latest_lat = float(msg.latitude)
        self.latest_lon = float(msg.longitude)
    except Exception:
        return
```

7.4 L111 関数 `WeatherFetchNode._fetch_once`

- 行範囲: L111–L135
- 所属: `WeatherFetchNode`
- 定義: `_fetch_once(self) -> None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `String`, `error`, `fetch_openmeteo_forecast`, `get_logger`, `info`, `is_absolute`, `len`, `publish`, `resolve`, `write_forecast_csv`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `df`, `lat`, `lon`, `raw_path`, `status`
- 読み取る主なインスタンス属性: `self.fallback_lat`, `self.fallback_lon`, `self.forecast_days`, `self.get_logger`, `self.latest_lat`, `self.latest_lon`, `self.output_csv`, `self.pub_status`, `self.raw_forecast_csv`, `self.step_minutes`, `self.tcell_gain`, `self.timezone_name`

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `lat` に `self.latest_lat` if `self.latest_lat` is not `None` else `self.fallback_lat` の結果を代入する。
2. `lon` に `self.latest_lon` if `self.latest_lon` is not `None` else `self.fallback_lon` の結果を代入する。
3. 例外処理を伴う try ブロックを実行する。
4. `df` に `fetch_openmeteo_forecast(lat, lon, timezone_name=self.timezone_name, forecast_days=self.forecast_days, step_minutes=self.step_minutes, tcell_gain=self.tcell_gain)` の結果を代入する。
5. `write_forecast_csv(...)` を実行する。
6. 条件 `self.raw_forecast_csv` を判定し、真なら内部処理を行う。
7. `raw_path` に `self.raw_forecast_csv` の結果を代入する。
8. 条件 `not raw_path.is_absolute()` を判定し、真なら内部処理を行う。
9. `raw_path` に `(self.output_csv.parent / raw_path).resolve()` の結果を代入する。
10. `write_forecast_csv(...)` を実行する。
11. `status` に `f'ok lat={lat:.5f} lon={lon:.5f} rows={len(df)} out={self.output_csv}'` の結果を代入する。
12. `self.pub_status.publish(...)` を実行する。
13. `self.get_logger().info(...)` を実行する。
14. Exception を捕捉した場合:
15. `status` に `f'error weather fetch failed: {exc}'` の結果を代入する。
16. `self.pub_status.publish(...)` を実行する。
17. `self.get_logger().error(...)` を実行する。

代表コード断片

```
def _fetch_once(self) -> None:
    lat = self.latest_lat if self.latest_lat is not None else self.fallback_lat
    lon = self.latest_lon if self.latest_lon is not None else self.fallback_lon
    try:
        df = fetch_openmeteo_forecast(
            lat,
            lon,
            timezone_name=self.timezone_name,
            forecast_days=self.forecast_days,
            step_minutes=self.step_minutes,
            tcell_gain=self.tcell_gain,
```

```

    )
    write_forecast_csv(df, self.output_csv)
    if self.raw_forecast_csv:
        raw_path = self.raw_forecast_csv
        if not raw_path.is_absolute():
            raw_path = (self.output_csv.parent / raw_path).resolve()
        write_forecast_csv(df, raw_path)
    status = f"ok lat={lat:.5f} lon={lon:.5f} rows={len(df)} out={self.output_csv}"
    self.pub_status.publish(String(data=status))
    self.get_logger().info(status)
except Exception as exc:
    status = f"error weather fetch failed: {exc}"
    self.pub_status.publish(String(data=status))
    self.get_logger().error(status)

```

7.5 L138 関数 main

- 行範囲: L138–L143
- 所属: module 直下
- 定義: `main()` → None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `WeatherFetchNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `rcipy.init(...)` を実行する。
2. `node` に `WeatherFetchNode()` の結果を代入する。
3. `rcipy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rcipy.shutdown(...)` を実行する。

代表コード断片

```

def main() -> None:
    rcipy.init()
    node = WeatherFetchNode()
    rcipy.spin(node)
    node.destroy_node()
    rcipy.shutdown()

```


7.6 設定値と入力パラメータ

行	名前	既定値・補足
18	profile_yaml	
19	output_csv	
20	raw_forecast_csv	
21	gps_topic	/chase/gps
22	fetch_period_sec	3600.0
23	forecast_days	3
24	step_minutes	10
25	timezone_name	Australia/Darwin
26	fallback_latitude	-12.4634
27	fallback_longitude	130.8456
28	tcell_gain	0.03

7.7 ROS topic I/O

行	種別	topic	callback / 補足
98	publisher	/weather/fetch_status	publisher
99	subscription	self.gps_topic	self._on_gps

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/weather_fetch_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。